Singly Linked List – Traverse List Inorder and Time Complexity.

Traversing List InOrder means traversing list starting from first index to last index.

Program:

```
void TraverseListInorder(Node **head)
{
    if (*head == nullptr)
    {
        cout << "List is empty." << endl;
        return;
    }
    else
    {
        Node *current = *head;
        while (current != nullptr)
        {
            cout << current->data << " ";
            current = current->next;
        }
        cout << endl;
    }
}
….
case 22:
        {
            cout << endl;
            cout << endl;
            cout << "Traverse List In Order 1:(While Loop)" << endl;
            TraverseListInorder(&head);
            cout << endl;
            cout << endl;
            break;
        }
```

Program Analysis:

Function Call:

$Function\ call\ occurs\ when\ Case\ 21\ is\ initiated\ from\ Switch - Case$:

$cout\ \ldots\ldots TraversingListInorder\ (\&head);$

$There\ are\ two\ criteria\ of\ input\ Linked\ List$:

1)*Linked List is Empty*:

$Pictorially\ depicted\ as$:

$Node * head$

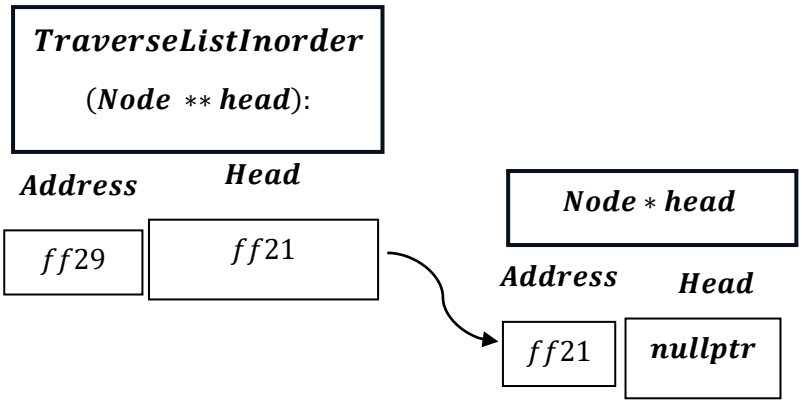
$Address$	$Head$
$ff21$	$nullptr$

```
    if (*head == nullptr)
    {
        cout << "List is empty." << endl;
        return;
    }
```

$TraverseListInorder(\&head: ff21);$

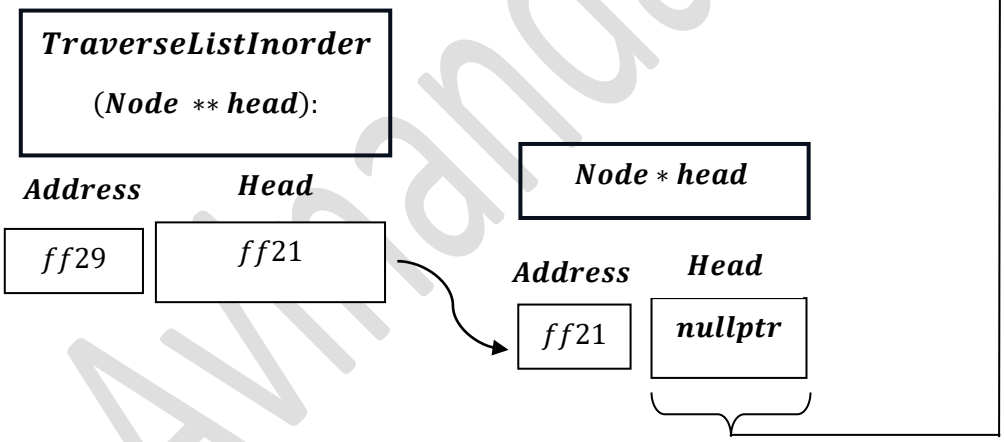
```
void TraverseListInorder(Node **head) {….}
```

Avinandan Bose



if $(*\ head == nullptr)$ ***if it is true***

If true then Why true ?
Ans: Though * head which is of Node ** head temporary pointer to pointer variable of the function , as * head is used , it dereference from Node ** head and refer to Node * head global pointer to which Node ** head is pointing to and then its value is checked is whether equal to nullptr or not ? This occurs during execution and compilation of the `TraverseListInorder` function.

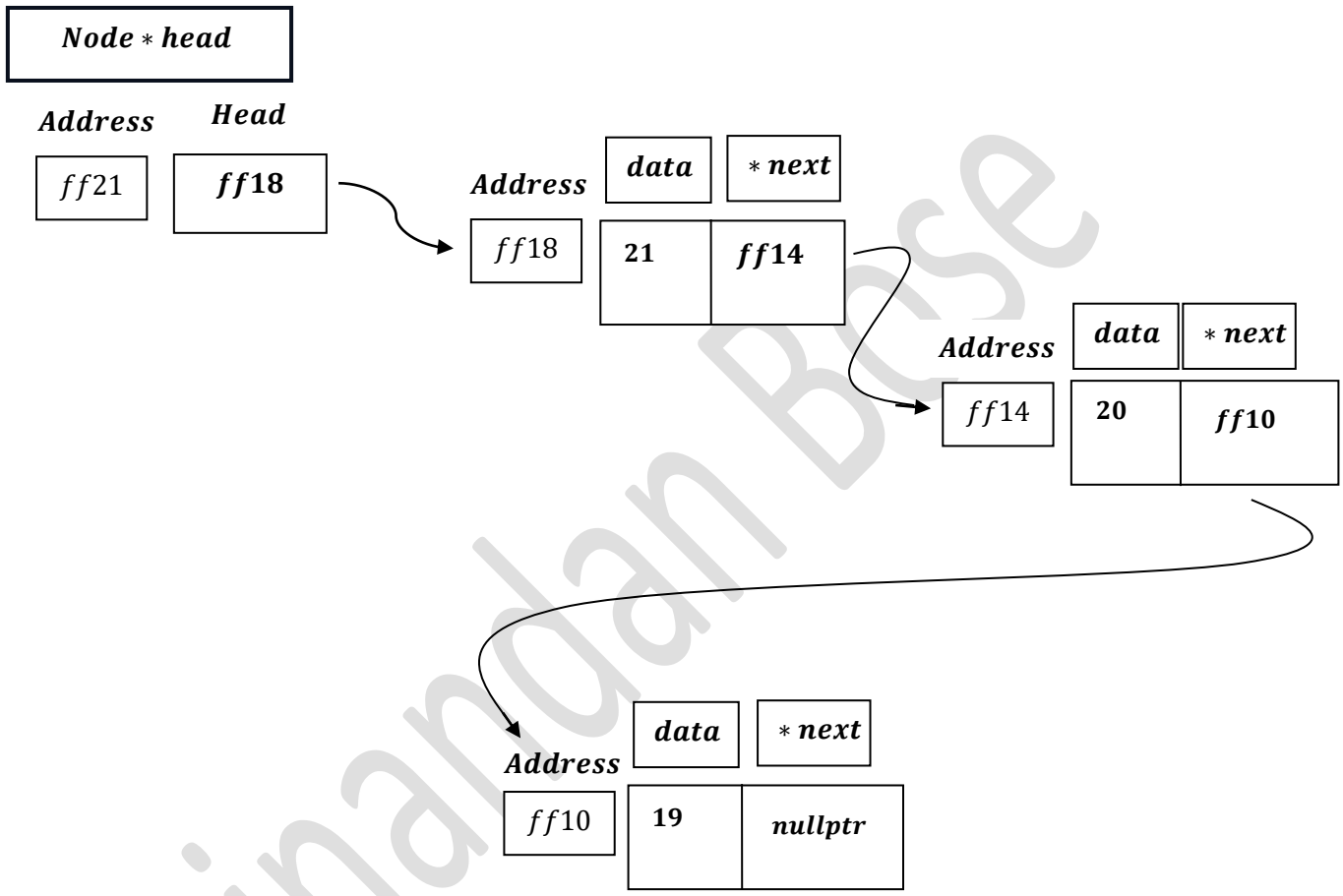


If List is Empty , then , it prints a message : `Error: List is empty.` and returns nothing as function has void data type . And function ends.

Avinandan Bose

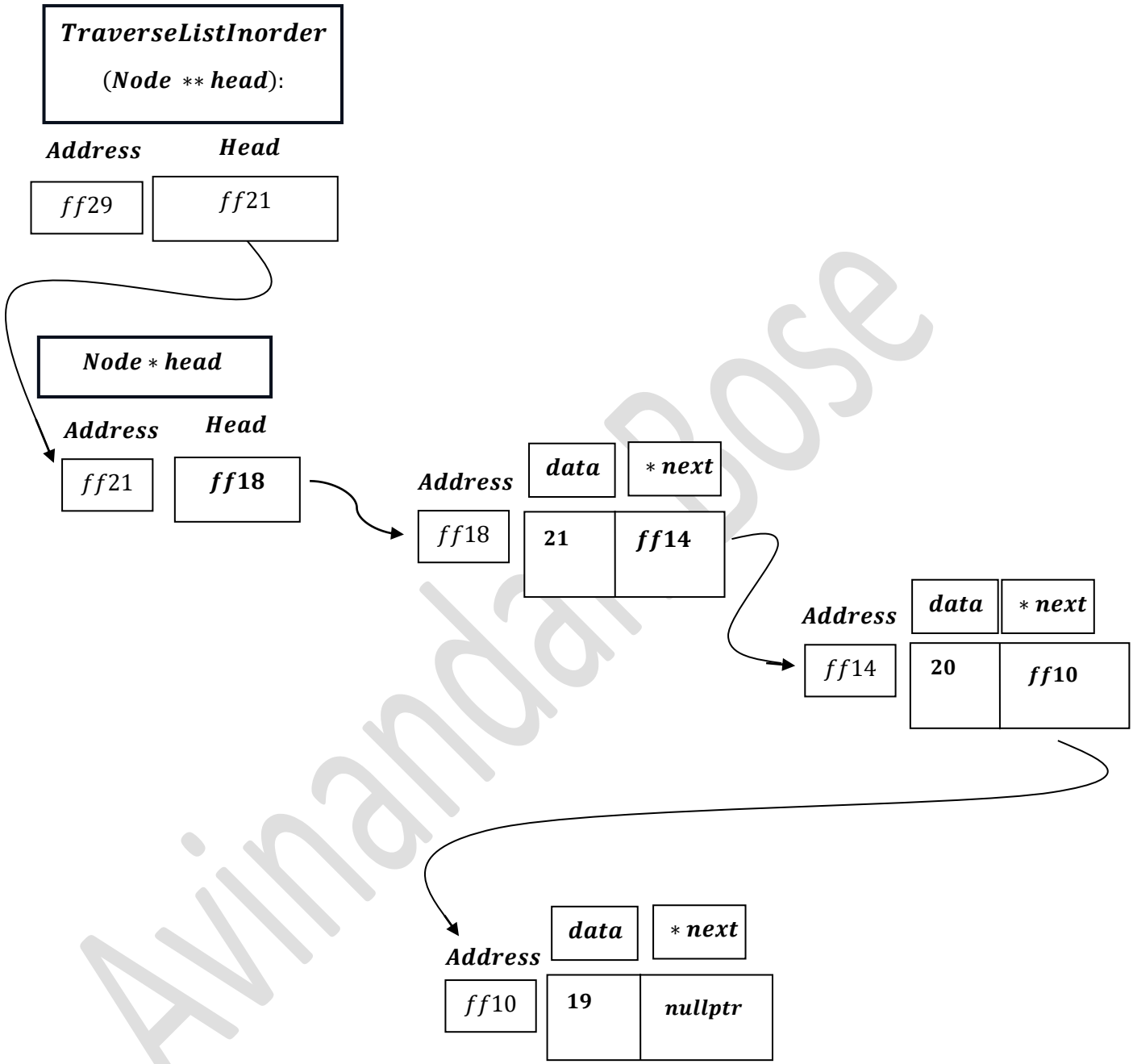
2)*Linked List is not Empty*:

Pictorially depicted as:



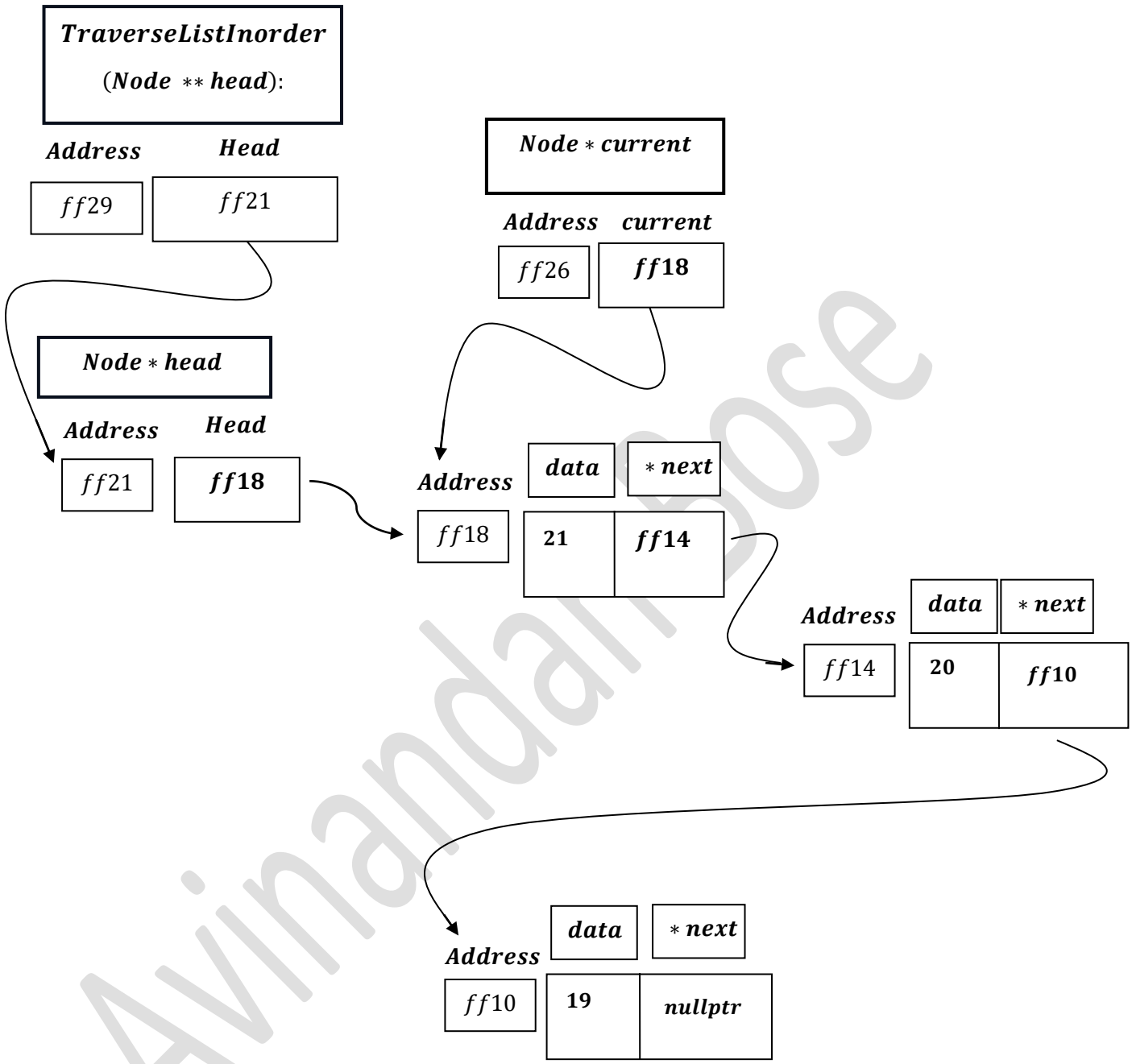

```
void TraverseListInorder(Node **head)
{ ……
    else
    {
        Node *current = *head;
        while (current != nullptr)
        {
            cout << current->data << " ";
            current = current->next;
        }
        cout << endl;
    }
}
```

```
void TraverseListInorder(Node **head){….}
```




```
        Node *current = *head;
```

Avinandan Bose



Now, we have while loop running :

Avinandan Bose

```
        while (current != nullptr)
        {
            cout << current->data << " ";
            current = current->next;
        }
```

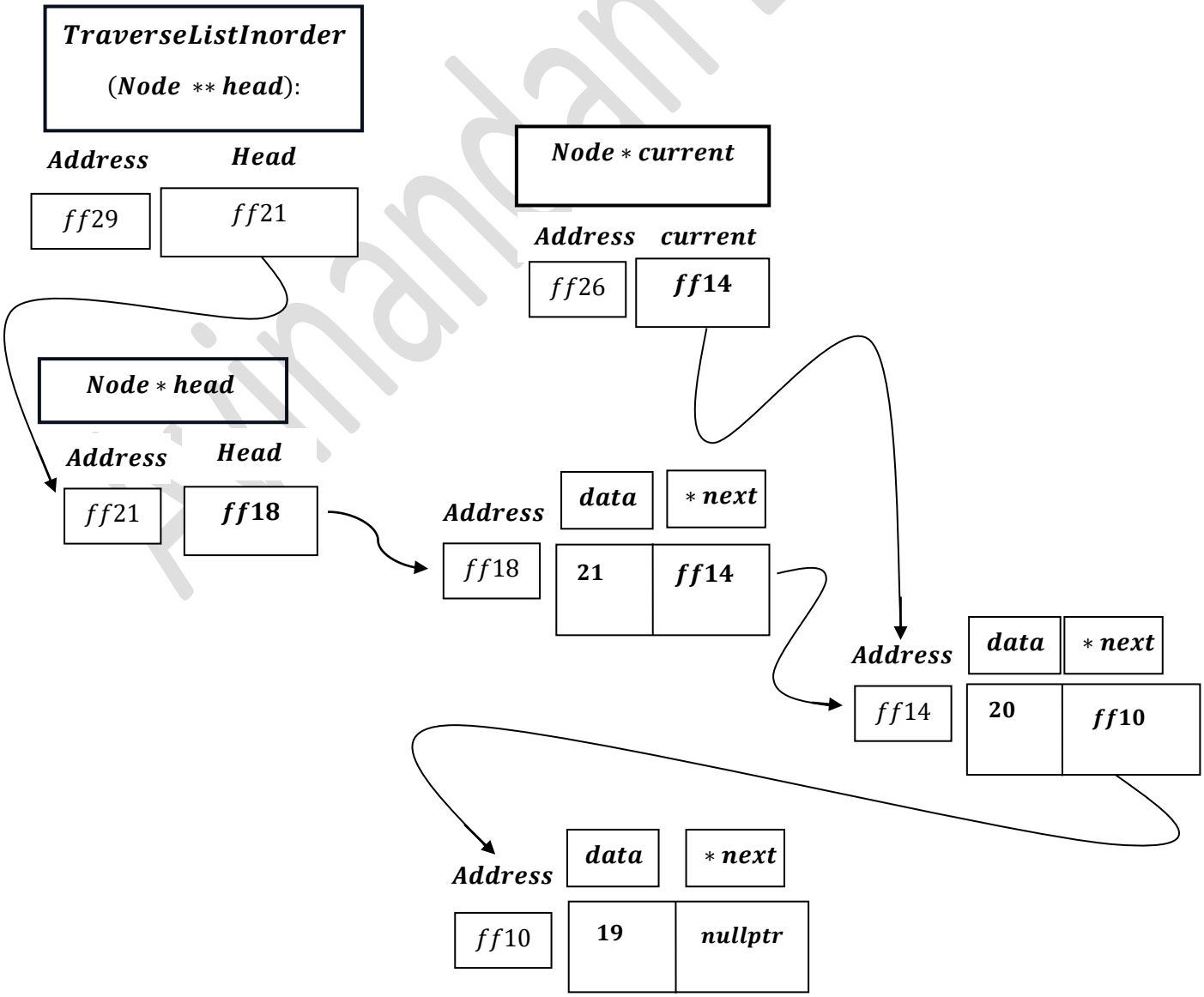
$Therefore, on\ first\ run, we\ have:$

$while(current != nullptr)\{$

$\quad cout \ll current \rightarrow data \ll ``\ ";\ Output: 21$

$\quad current = current \rightarrow next;\ i.e.\ current = ff14$

$\}$



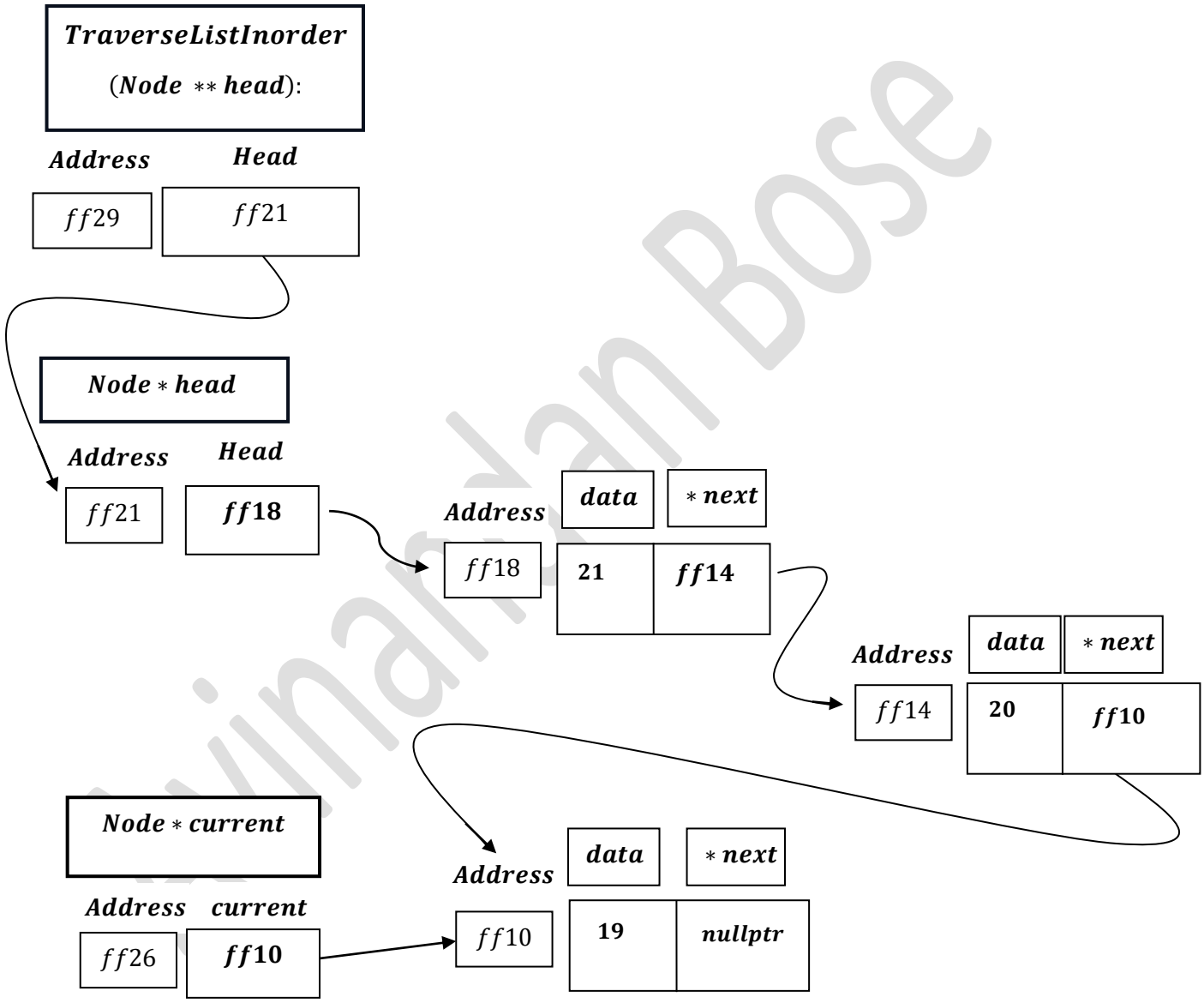
Avinandan Bose

$$while(current\ !=\ nullptr)\{$$

$$cout \ll current \rightarrow data \ll \text{``}\ \text{''}; Output : 20$$

$$current = current \rightarrow next;\ i.e. current = ff10$$

$$\}$$



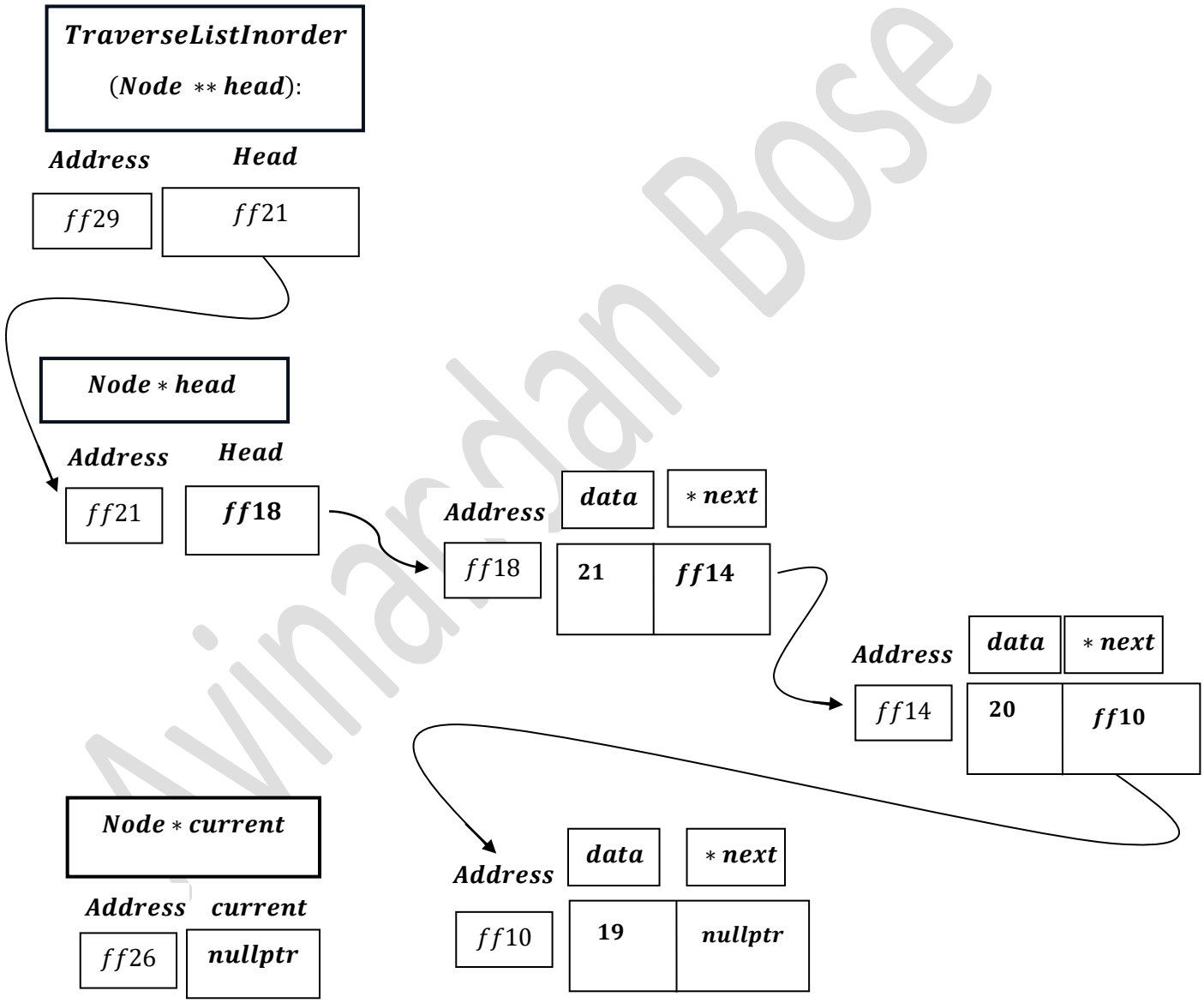
Avinandan Bose

$while(current\ != nullptr)\{$

$\quad cout \ll current \rightarrow data \ll \text{``}\ \text{''}; Output : 19$

$\quad current = current \rightarrow next;\ i.e. current = nullptr$

$\}$

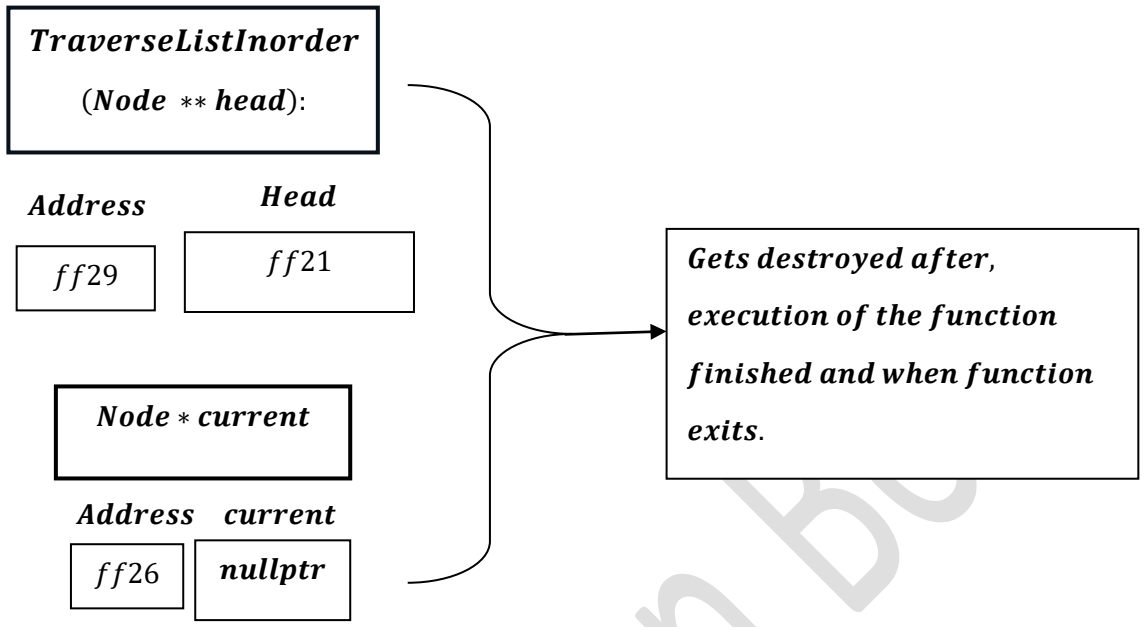


$Now, as\ soon\ as\ , current\ becomes\ \text{``}nullptr\text{''}\ , the\ condition\ of\ while\ loop\ breaks$

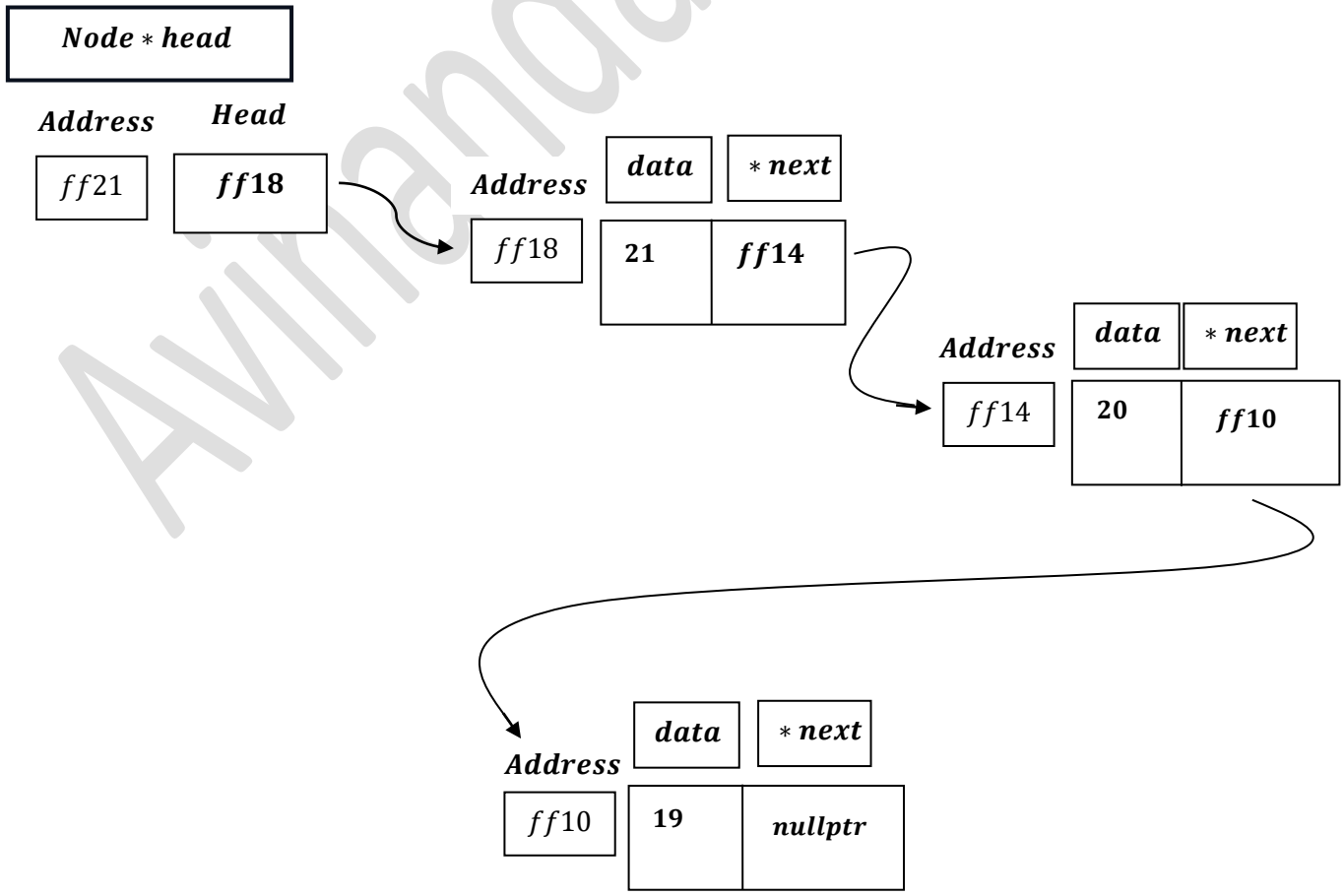
$i.e. while(current\ != nullptr)\ becomes\ false\ and\ loop\ ends.$

Avinandan Bose

Now, as the function get finished, all temporary variable created earlier in process for the program will get destroyed.



And we are left with:



Avinandan Bose

Now, `TraverseListInorder` step by step explanation :

1. *Check if the List is Empty*:

- *The function first checks if the list is empty by verifying if the head pointer is nullptr.*
- *If the list is empty, it prints* List is empty.*and returns immediately, as there are no nodes to traverse.*

2. *Initialize Current Pointer*:

- *A pointer current is initialized to point to the head of the list. This pointer will be used to traverse the list.*

3. *Traverse the List*:

- *The function enters a while loop that continues as long as current is not nullptr.*
- *Inside the loop, it prints the data of the current node followed by a space.*
- *It then moves the current pointer to the next node in the list using current = current−> next.*

4. *Print Newline*:

- *After the loop completes (i.e., when current becomes nullptr), the function prints a newline character to ensure the output is neatly formatted.*

Avinandan Bose

Time Complexity of
Traverse List Inorder

```
void TraverseListInorder(Node **head)
{
    if (*head == nullptr)
    {
        cout << "List is empty." << endl;
        return;
    }
    else
    {
        Node *current = *head;
        while (current != nullptr)
        {
            cout << current->data << " ";
            current = current->next;
        }
        cout << endl;
    }
}
```

$O(1)$

$O(1)$

Loop runs n times, hence statements runs: $n + n = 2n = 2 \times O(n) = O(n)$.

$O(1)$

Best Case (When List is Empty)

When List is Empty, then, as soon as it enters the function: TraverseListInorder, if checks whether head pointer pointed to ``nullptr`` or not , if so then outputs: ``List Is Empty`` and returns nothing , and then exits from function. This takes, constant `c` amount of time i.e. $\Omega(1)$.

Hence when List is Empty, we get our best case : $\Omega(1)$.

$\underline{Worst\ Case\ (When\ List\ Is\ Not\ Empty)}$

$When\ list\ is\ not\ empty\ ,it\ traverse\ whole\ list\ ,assuming\ `n`\ is\ the\ length\ of\ the\ List, then\ the\ loop\ traverse\ `n`\ no.of\ nodes\ present\ in\ the\ list.$

$Where\ statement: cout \ll current \rightarrow data \ll ``\ ``; outputs\ ,`n`\ times.$
$current = current \rightarrow next; runs\ `n`\ times\ .$

$Hence\ total\ number\ of\ statements\ in\ the\ loop\ runs: n + n = 2n = 2 \times O(n) = O(n).$

$\underline{Average\ Case(Average\ Case = Worst\ Case)}$

$The\ average\ case\ for\ traversing\ a\ linked\ list\ typically\ involves\ traversing\ a\ list\ of\ average\ length. Since\ the\ function\ must\ visit\ each\ node\ to\ print\ its\ data, the\ time\ complexity\ is\ (O(n)), where\ (n)\ is\ the\ number\ of\ nodes\ in\ the\ list.$

$The\ average\ case\ for\ traversing\ a\ linked\ list\ typically\ involves\ traversing\ a\ list\ of\ average\ length. Since\ the\ function\ must\ visit\ each\ node\ to\ print\ its\ data, the\ time\ complexity\ is\ (O(n)), where\ (n)\ is\ the\ number\ of\ nodes\ in\ the\ list.$

$The\ average\ case\ for\ traversing\ a\ linked\ list\ typically\ involves\ traversing\ a\ list\ of\ average\ length. Since\ the\ function\ must\ visit\ each\ node\ to\ print\ its\ data, the\ time\ complexity\ is\ (O(n)), where\ (n)\ is\ the\ number\ of\ nodes\ in\ the\ list.$

$Note\ ,by\ relation: \Omega \leq \Theta \leq O\ ,we\ get:$
$\Rightarrow c_1 \times g(n) \leq f(n) \leq c_2 \times g(n)$
$\Rightarrow c_1 \times 1 \leq n \leq c_2 \times n$

Avinandan Bose

Singly Linked List	*Cases*	*Time Complexity*
Traversal List Inorder	1. *Best Case*	$\Omega(1)$
	2. *Worst Case*	$O(n)$
	3. *Average Case = Worst Case*	$\Theta(n)$

****

Avinandan Bose